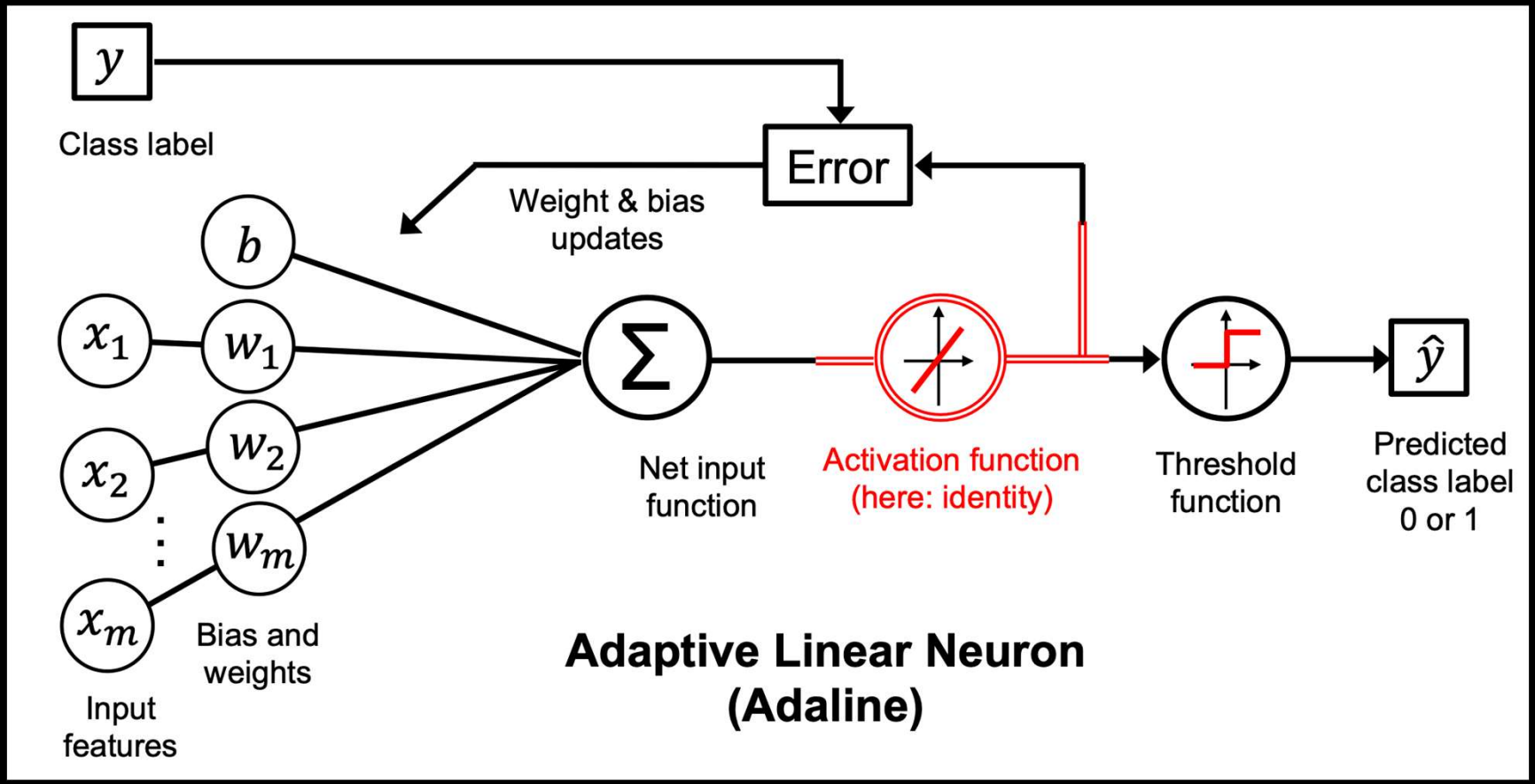
Neural Networks and Deep Learning

Dr.Varodom Toochinda
EMME, Kasetsart University

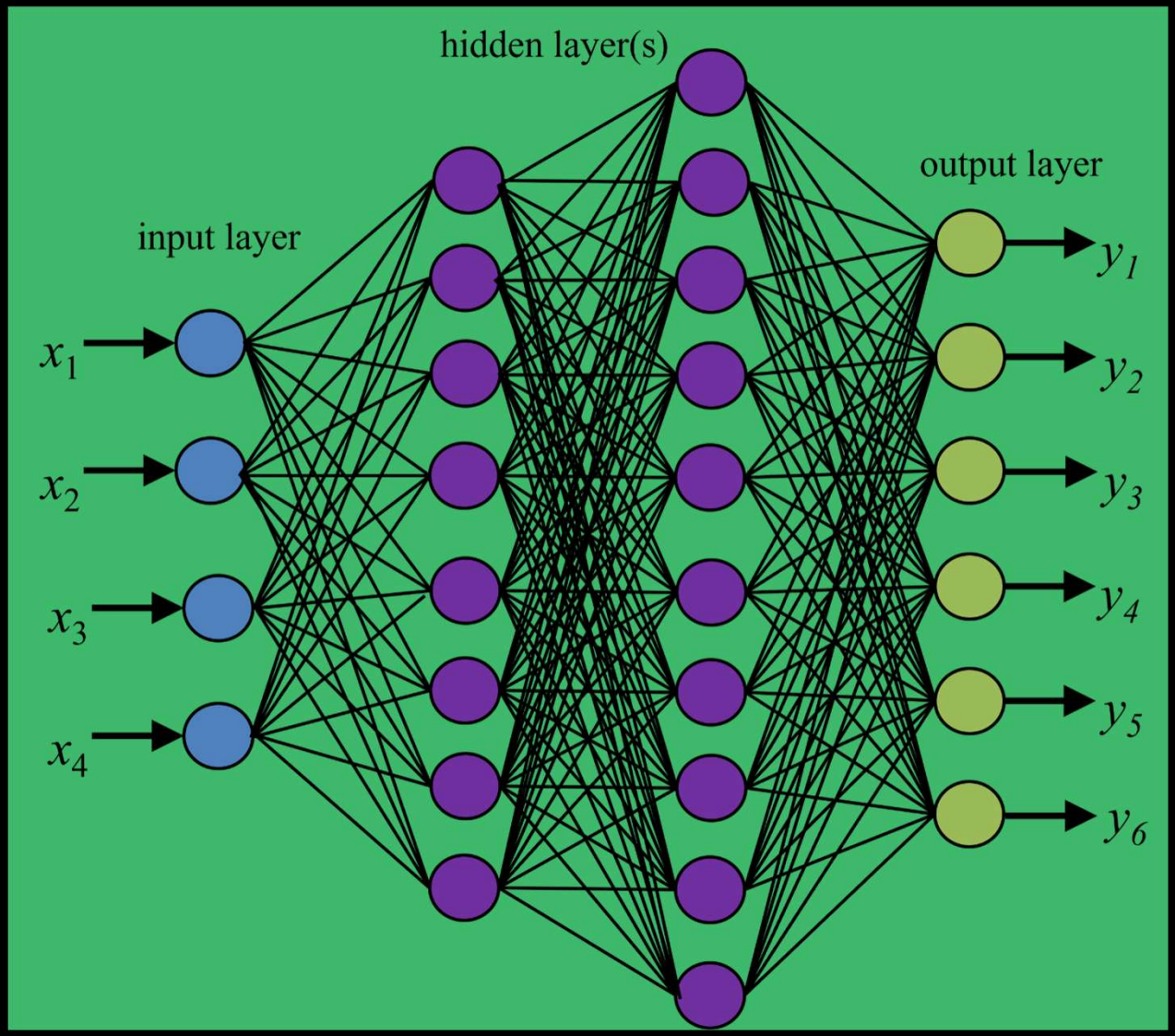
Topics

- Implementing an NN model from scratch
- Building an NN model in PyTorch
- The mechanics of PyTorch (optional)

Recall the Adaline



Multi-layer NN (MLP or DNN)

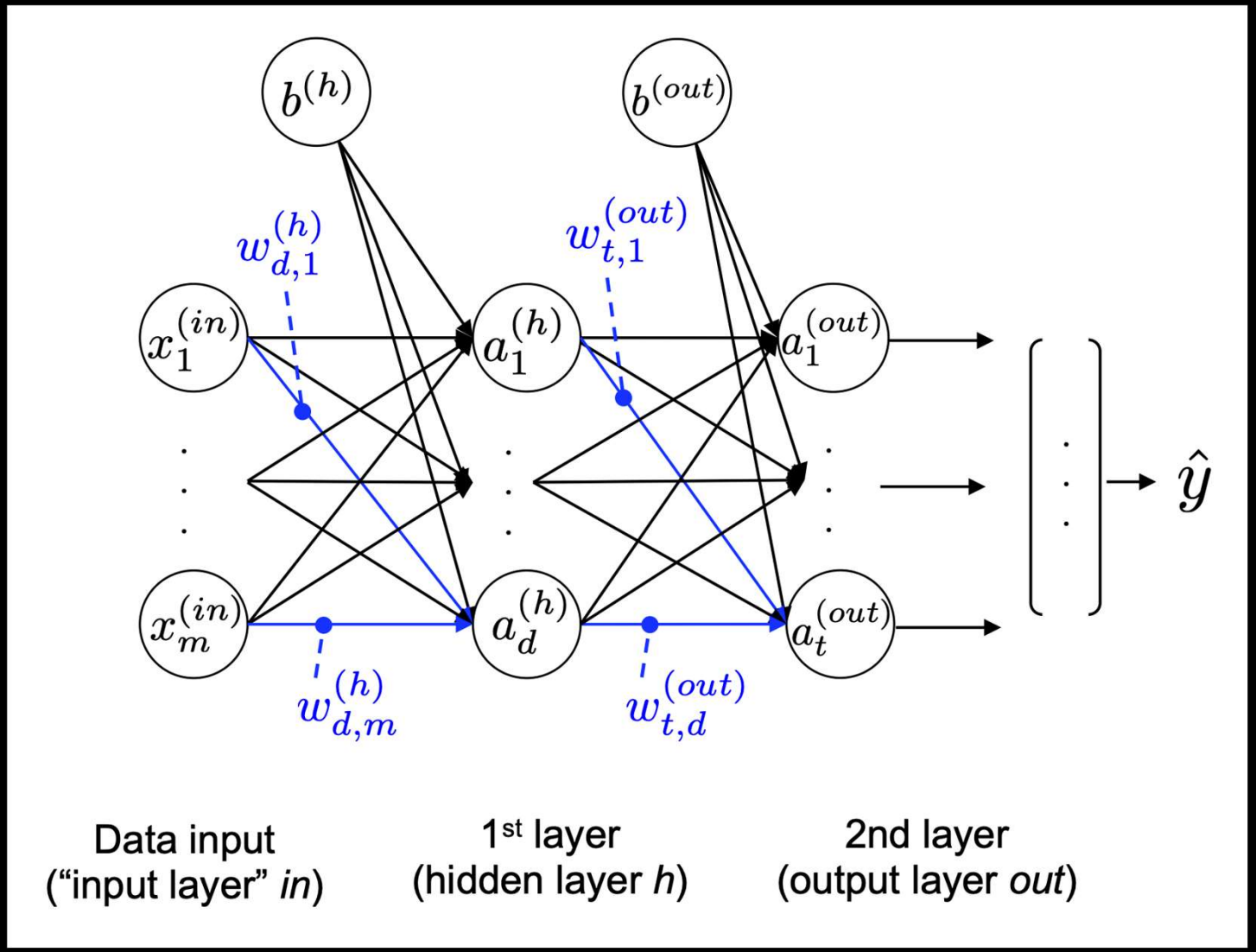


Universal Approximation Theorem

In the field of machine learning, the universal approximation theorems state that neural networks with a certain structure can, in principle, approximate any continuous function to any desired degree of accuracy. These theorems provide a mathematical justification for using neural networks, assuring researchers that a sufficiently large or deep network can model the complex, non-linear relationships often found in real-world data.

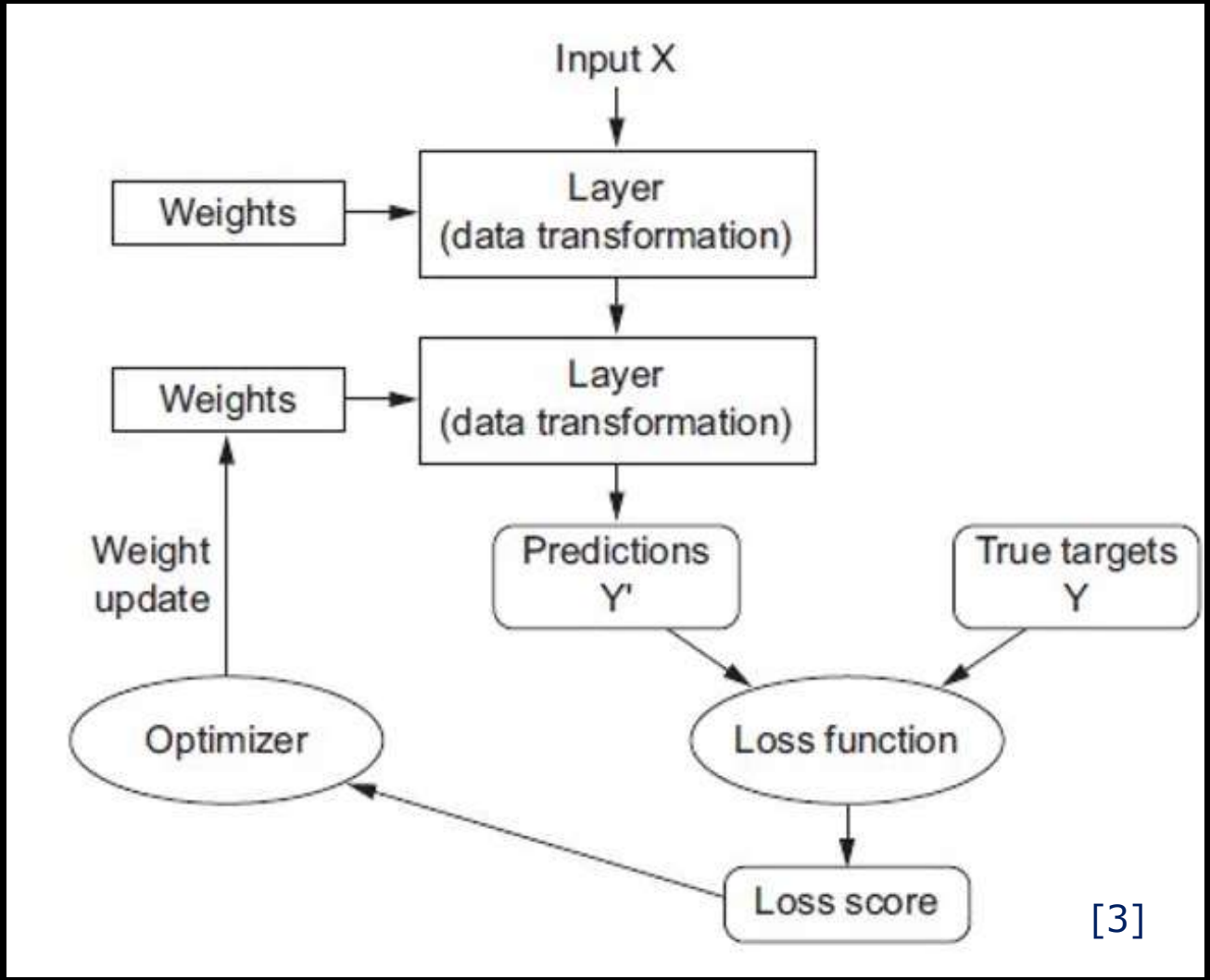
https://en.wikipedia.org/wiki/Universal_approximation_theorem

A two-layer MLP



From
Figure 11.2 of [1]

MLP learning structure



[3]

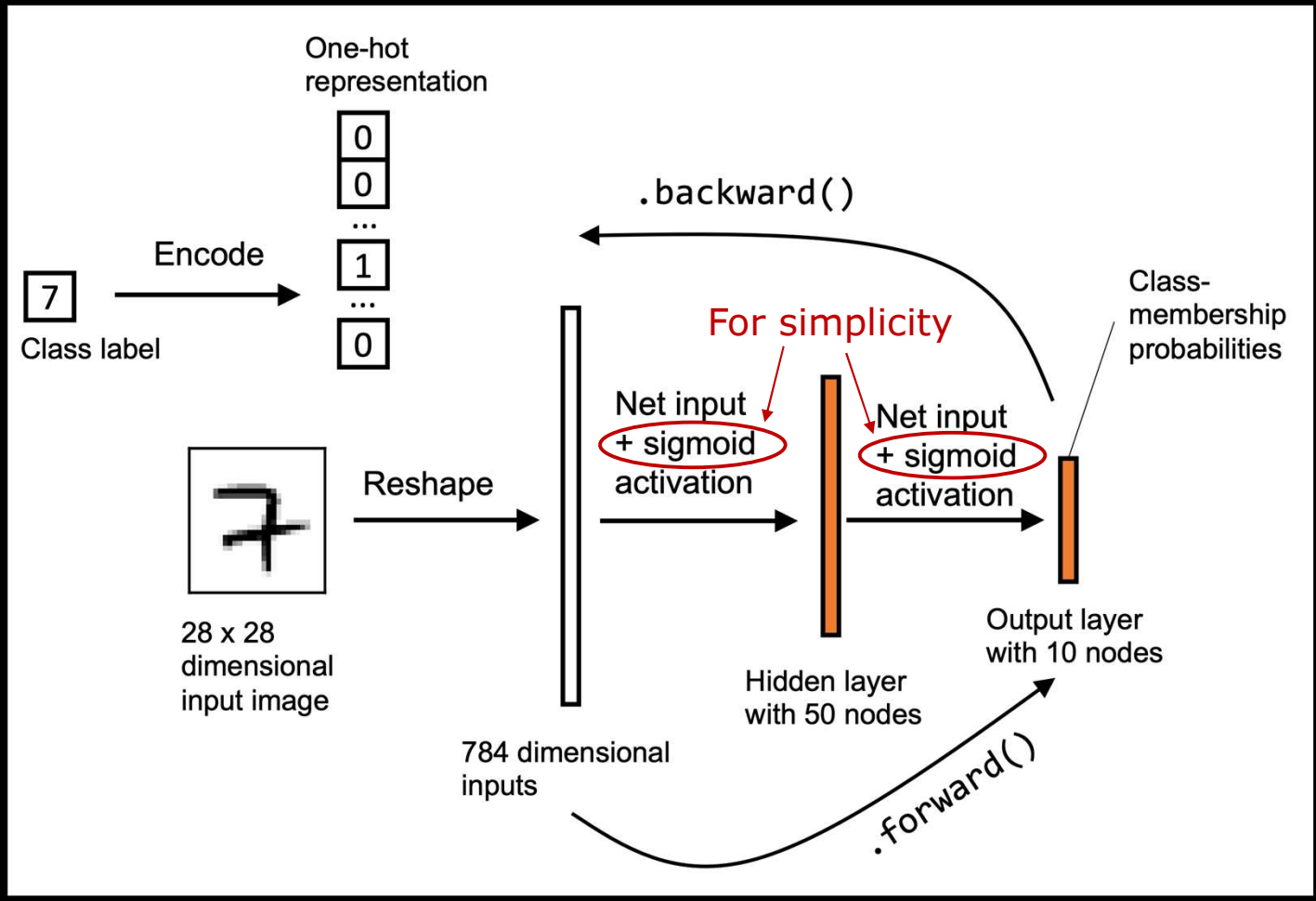
Classifying MNIST hand-written digits

Run code in mlpi25_lect09.ipynb

Implementing an MLP

- See NeuralNetMLP class in mlpi25_lect09.ipynb
- 3 class methods:
  - __init__() # constructor
  - forward() # prediction
  - backward() # compute gradients
- For simplicity, MSE loss is used in backward()
- Detailed explanation in [1]

NN architecture for lebeling hand-written digits



From
Figure 11.6 of [1]

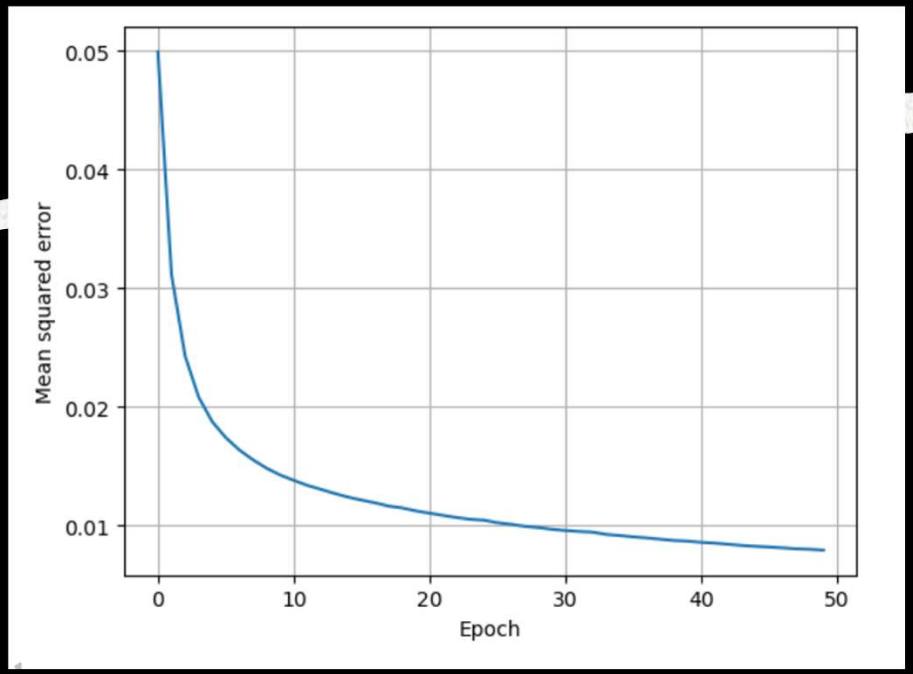
Training the MLP

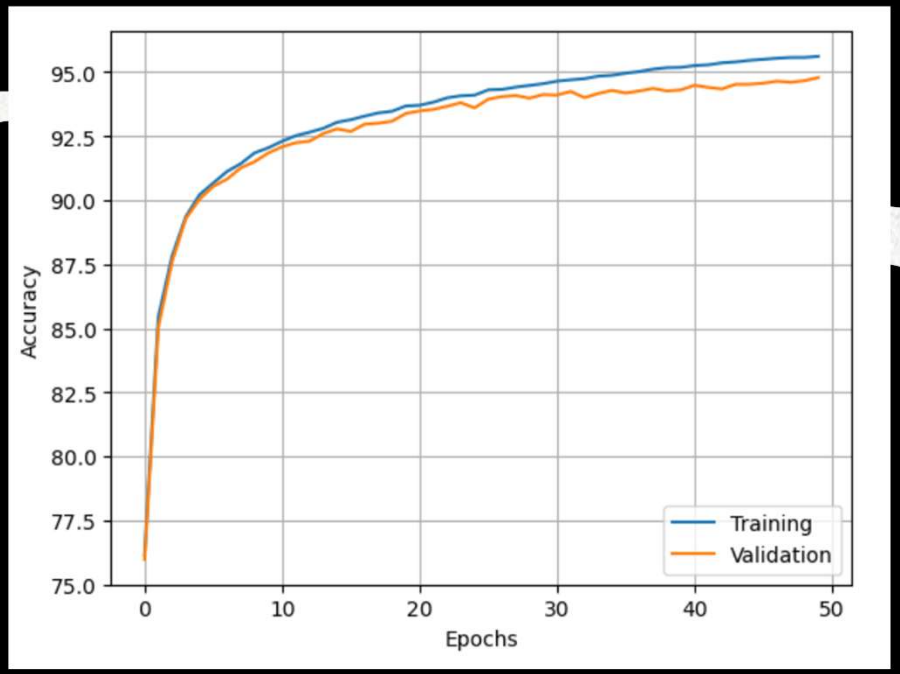
```
epoch_loss, epoch_train_acc, epoch_valid_acc = train(
    model, X_train, y_train, X_valid, y_valid,
    num_epochs=50, learning_rate=0.1)
```

[16] 2m 10.2s

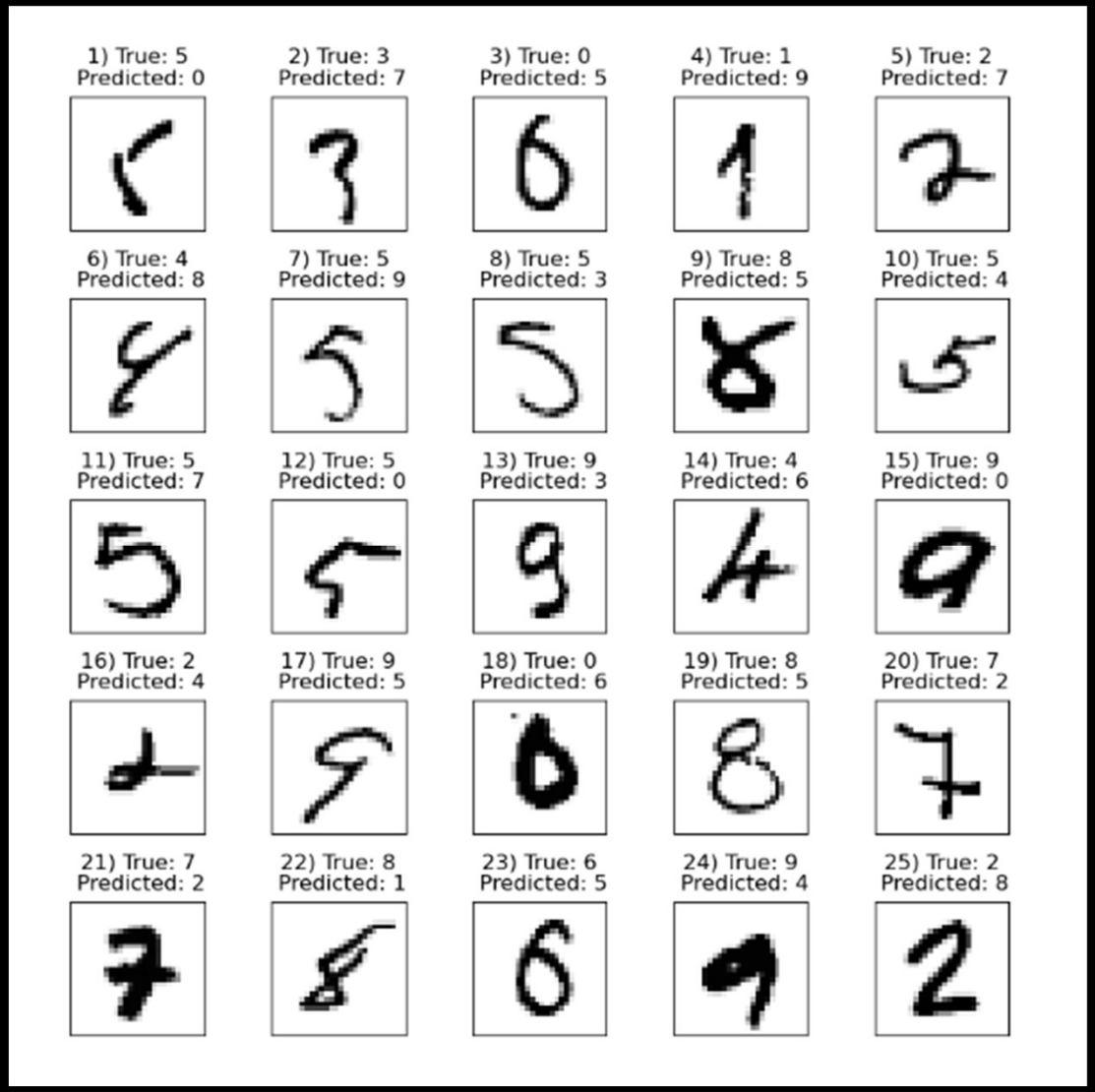
```
Epoch: 001/050 | Train MSE: 0.05 | Valid MSE: 0.05 | Train Acc: 76.15% | Valid Acc: 75.98%
Epoch: 002/050 | Train MSE: 0.03 | Valid MSE: 0.03 | Train Acc: 85.45% | Valid Acc: 85.04%
Epoch: 003/050 | Train MSE: 0.02 | Valid MSE: 0.02 | Train Acc: 87.82% | Valid Acc: 87.60%
Epoch: 004/050 | Train MSE: 0.02 | Valid MSE: 0.02 | Train Acc: 89.36% | Valid Acc: 89.28%
Epoch: 005/050 | Train MSE: 0.02 | Valid MSE: 0.02 | Train Acc: 90.21% | Valid Acc: 90.04%
Epoch: 006/050 | Train MSE: 0.02 | Valid MSE: 0.02 | Train Acc: 90.67% | Valid Acc: 90.54%
Epoch: 007/050 | Train MSE: 0.02 | Valid MSE: 0.02 | Train Acc: 91.12% | Valid Acc: 90.82%
Epoch: 008/050 | Train MSE: 0.02 | Valid MSE: 0.02 | Train Acc: 91.43% | Valid Acc: 91.26%
Epoch: 009/050 | Train MSE: 0.01 | Valid MSE: 0.01 | Train Acc: 91.84% | Valid Acc: 91.5
Epoch: 010/050 | Train MSE: 0.01 | Valid MSE: 0.01 | Train Acc: 92.04% | Valid
Epoch: 011/050 | Train MSE: 0.01 | Valid MSE: 0.01 | Train Acc: 92.30% | Vali
Epoch: 012/050 | Train MSE: 0.01 | Valid MSE: 0.01 | Train Acc: 92.51% |
```

Evaluating NN performance



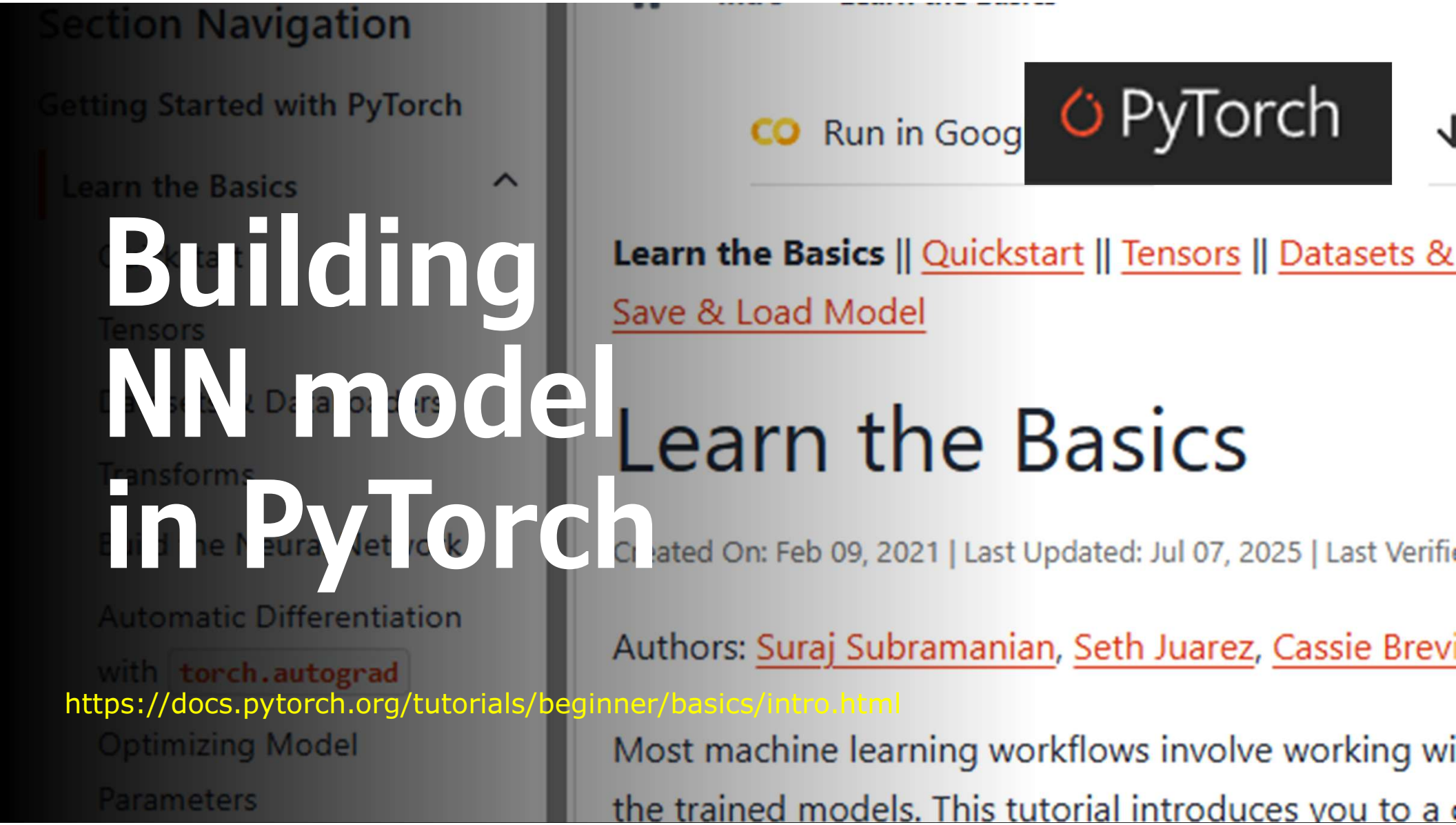


Misclassification images



Optional reading for Chapter 11 of [1]

- Training an artificial neural network (p.360)
  - Computing the loss function
  - Developing your understanding of backpropagation
  - Training neural networks via backpropagation
  - About convergence in neural networks



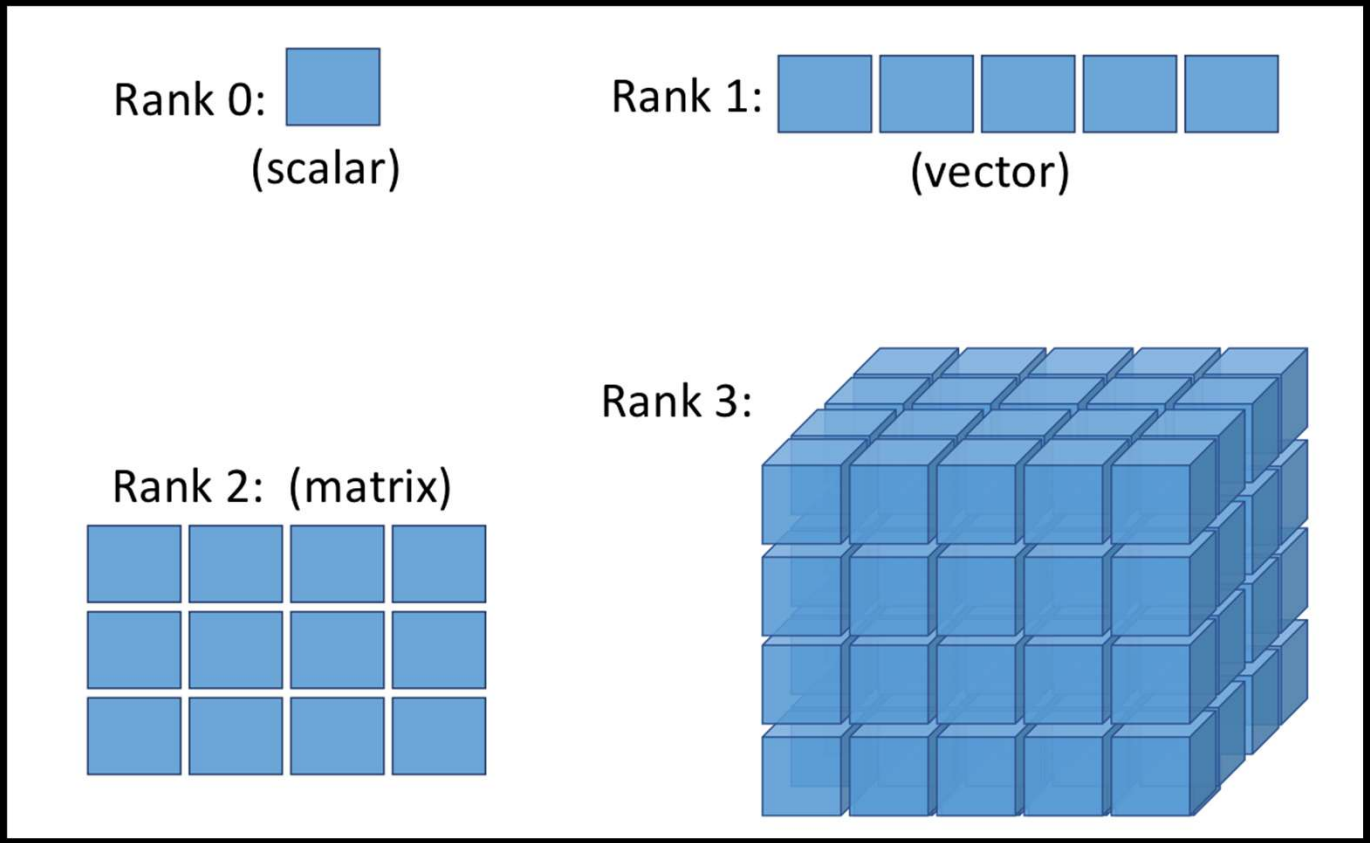
Building NN model in PyTorch

Topics

- Tensors in PyTorch
- Building input pipelines
- Building an NN model
- Choosing activation functions in MLP
- The Mechanics of PyTorch (optional) see mlpi25_lect09A.ipynb

Ref: chapter 12 – 13 of [1]

Different types of tensor in PyTorch



Tensor creation/operations in PyTorch

- See mlpi25_lect09.ipynb

Building input pipelines in PyTorch

- Creating a PyTorch DataLoader
  - torch.utils.data.DataLoader()
- Combining two tensors into a joint dataset
  - torch.utils.data.TensorDataset()
- Shuffle, batch, and repeat
- Creating a dataset from files

Example images

Fetching from torchvision.datasets

- CelebA
  http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html
  - torchvison.datasets.CelebA
    https://pytorch.org/vision/stable/datasets.html#celeba

  Note : this dataset is large (1.44 G) and may cause technical difficulty downloading.
  See [1] for alternative way to get the files.

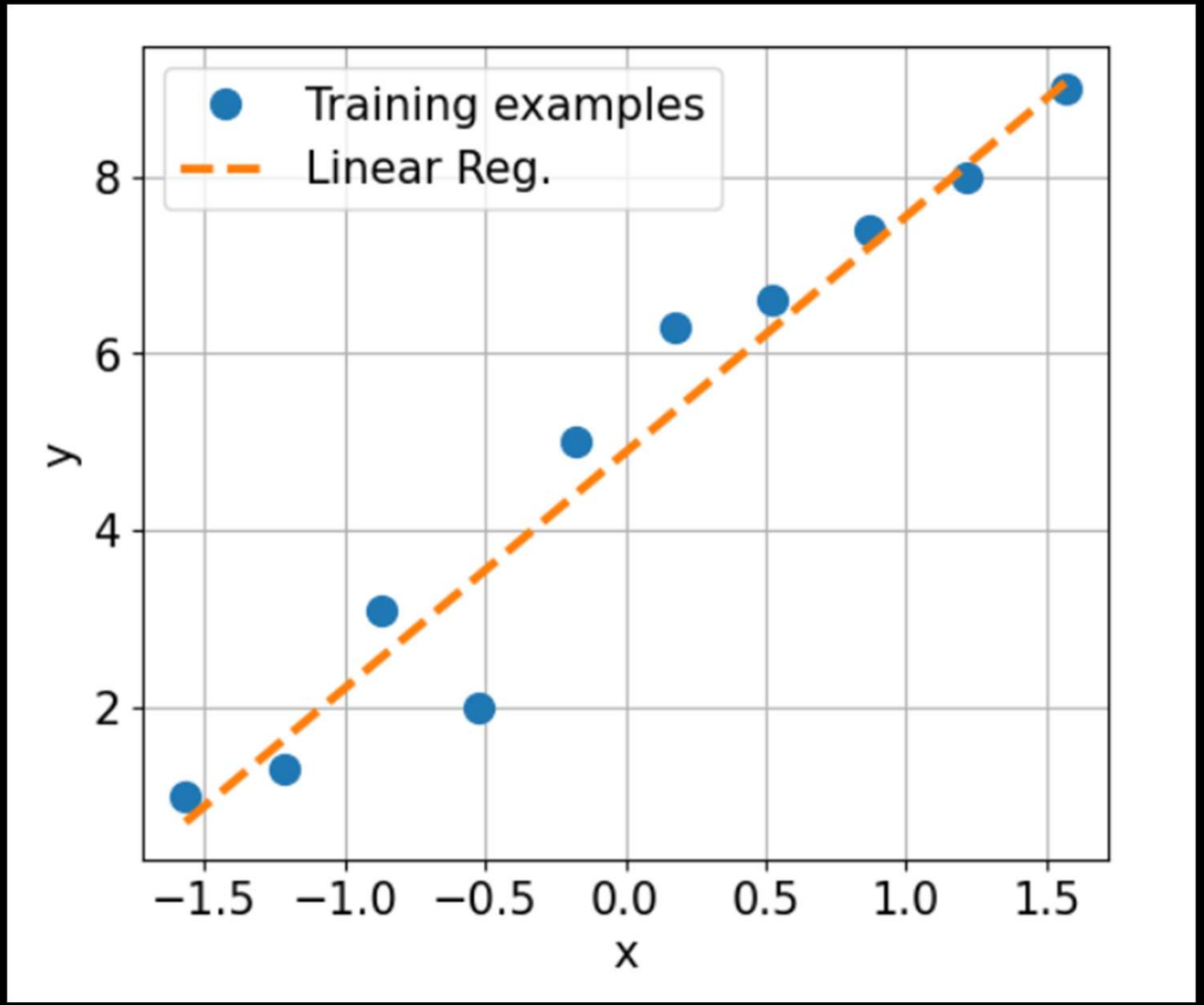
- MNIST
  - torchvison.datasets.MNIST
    https://pytorch.org/vision/stable/datasets.html#mnist

“Smiling” celeb

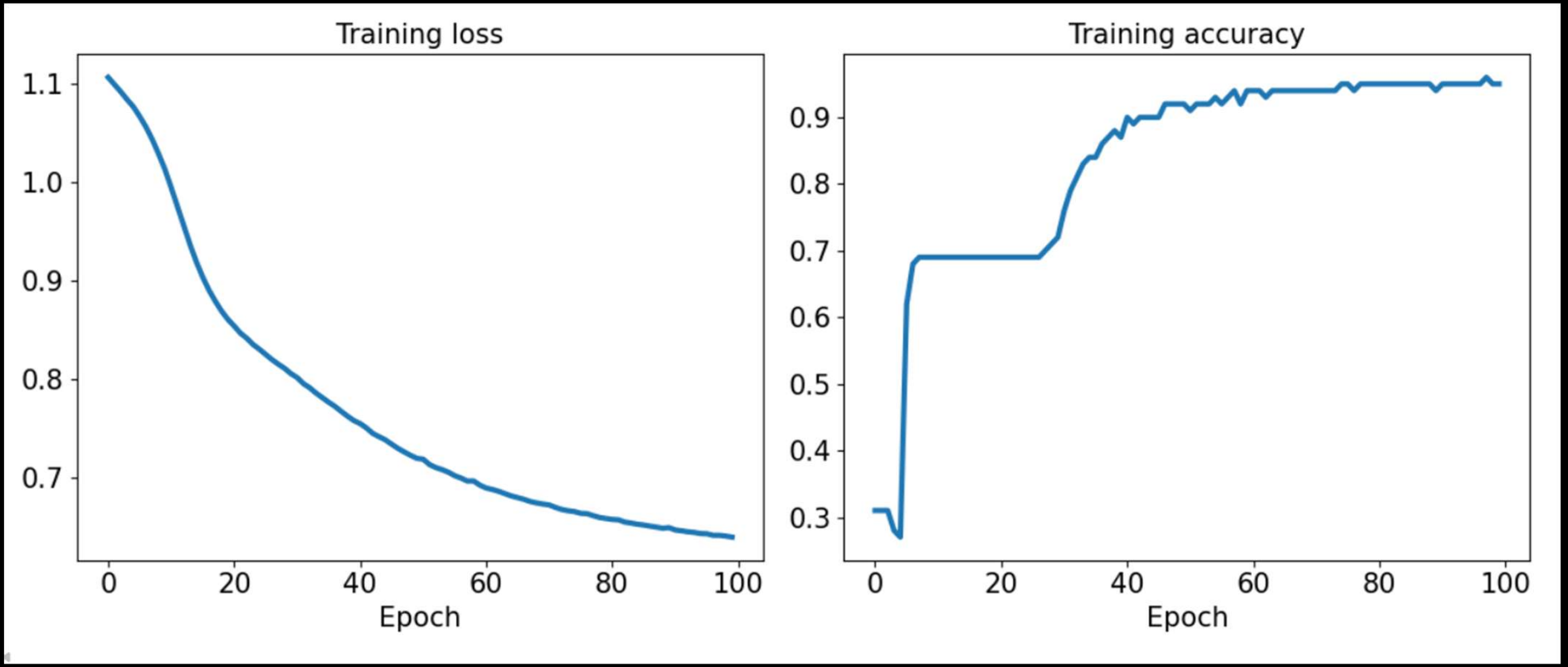
Building NN model

- linear regression
- MLP

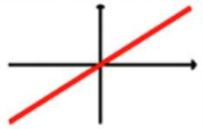
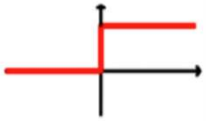
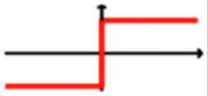
Linear regression model



MLP for iris dataset



Activation functions

Activation function	Equation	Example	1D graph
Linear	$\sigma(z) = z$	Adaline, linear regression	
Unit step (Heaviside function)	$\sigma(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (signum)	$\sigma(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Piece-wise linear	$\sigma(z) = \begin{cases} 0 & z \leq -½ \\ z + ½ & -½ \leq z \leq ½ \\ 1 & z \geq ½ \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\sigma(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, multilayer NN	
Hyperbolic tangent (tanh)	$\sigma(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multilayer NN, RNNs	
ReLU	$\sigma(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$	Multilayer NN, CNNs	

The mechanics of PyTorch (optional reading)

See chapter 13 of [1] and mlpi25_lect9A.ipynb

References

1. S. Raschka, Y. Liu and V. Mirjalili. Machine Learning with PyTorch and Scikit-Learn. https://sebastianraschka.com/blog/2022/ml-pytorch-book.html Packt Publishing. 2022.

2. A. Ng and T. Ma. CS229 Lecture Notes.https://cs229.stanford.edu/main_notes.pdf Stanford University. 2023.

3. F. Chollet. Deep Learning with Python 2ed. Manning Publications Co. 2021.

4. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-Based Learning Applied to Document Recognition , Proceedings of the IEEE, 86(11): 2278-2324, 1998.

5. C.M. Bishop. Neural Networks for Pattern Recognition, Oxford University Press, 1995.